Course Project

This course project is out of 100 points: 50 points for a lexer and 50 points for a parser, as detailed below. Please submit your solution to this project as a pair of python files `MiniCLexer.py` and `MiniCParser.py` for the lexer and parser, respectively. The main program `MiniC.py` which launches these is attached to this project—please use this, and make your lexer and parser work with `MiniC.py`.

Write a lexer and a parser using PLY (very much like that for the WAE language) for the simplified C language with the following grammar, with nonterminals surrounded by angular braces `< >`, the → is a production, and everything else is literal (terminal) symbols:

`<stmt>` → `<id> = <expr> ;`
`<stmt>` → `{ <stmtlist> }`
`<stmt>` → `if ( <expr> ) <stmt>`
`<stmt>` → `while ( <expr> ) <stmt>`

`<stmtlist>` → `<stmt>`
`<stmtlist>` → `<stmtlist> <stmt>`

`<expr>` → `<id>`
`<expr>` → `<num>`
`<expr>` → `<expr> <optr> <expr>`

where `<optr>` can be any operator from the set { `>=`, `<=`, `==`, `>`, `<`, `+`, `-`, `*`, `/` }, and `<id>` can be any legal identifier in most programming languages, such as `x`, `y`, `Xy`, `y_1`, `z2`, *etc.*, and `<num>` can be any real number, such as `8`, `-8`, `+9`, `9.`, `-9.`, `+10.5`, `3.14159`, *etc.*

Note that this grammar is very similar to that presented on the 6th slide titled "Example" of the slides on context-free languages on iCollege. In fact, this grammar is more general, in that it can do everything the grammar on this slide can do, but also has a while loop, a richer set of operators (than just `>` and `+`), can represent identifiers beyond just `x` and `y`, and can represent any real number (not just `0`, `1` and `9`).

You need only write a lexer and a parser (like with the WAE example), which outputs the "parse tree". For example, since this language is more general than that presented on the 6th slide, it can handle the input:

```
if(x > 9) { x = 0; y = y + 1; }
```

outputting parse tree:

```
['if', [['id', 'x'], ['optr', '>'], ['num', 9.0]], [['id', 'x'],
'=', ['num', 0.0], ['id', 'y'], '=', [['id', 'y'], ['optr', '+'], ['num', 1.0]]]]
```

but also something more general, like:

```
while(x == 5.5) { x = x + 3.5; y = y * 2.5; }
```

outputting parse tree:

```
['while', [['id', 'x'], ['optr', '=='], ['num', 5.5]],
[['id', 'x'], '=', [['id', 'x'], ['optr', '+'], ['num', 3.5]],
['id', 'y'], '=', [['id', 'y'], ['optr', '*'], ['num', 2.5]]]]
```

Of course, since an `if` or `while` passes control to some `<stmt>` underneath the `if` (or `while`) condition, this `<stmt>` could itself be an `if` or `while`, and so (arbitrary) nesting is possible:

```
while(i > 0) { if(j == 0) { x = x - 1; } }
```

outputting parse tree:

```
['while', [['id', 'i'], ['optr', '>'], ['num', 0.0]],
['if', [['id', 'j'], ['optr', '=='], ['num', 0.0]],
[['id', 'x'], '=', [['id', 'x'], ['optr', '-'], ['num', 1.0]]]]]
```

Since a `<stmtlist>` can be arbitrary in length, it is possible to have:

```
{ a = 1; b = 2; c = 3; d = 4; }
```

outputting parse tree:

```
[['id', 'a'], '=', ['num', 1.0], ['id', 'b'], '=', ['num', 2.0],
['id', 'c'], '=', ['num', 3.0], ['id', 'd'], '=', ['num', 4.0]]
```

and expressions (interleaved by operators) can also be arbitrary in length, and so it is possible to have:

```
a = b + c + d + x + y;
```

outputting parse tree:

```
[['id', 'a'], '=', [['id', 'b'], ['optr', '+'], [['id', 'c'], ['optr', '+'],
[['id', 'd'], ['optr', '+'], [['id', 'x'], ['optr', '+'], ['id', 'y']]]]]]
```

hence a combination of the two is possible:

```
{ a = a_1 + a_2 + a_3; b = b_1 + b_2 + b_3; c = c_1 + c_2 + c_3; }
```

outputting parse tree:

```
[['id', 'a'], '=', [['id', 'a_1'], ['optr', '+'], [['id', 'a_2'],
['optr', '+'], ['id', 'a_3']]],
['id', 'b'], '=', [['id', 'b_1'], ['optr', '+'], [['id', 'b_2'],
['optr', '+'], ['id', 'b_3']]],
['id', 'c'], '=', [['id', 'c_1'], ['optr', '+'], [['id', 'c_2'],
['optr', '+'], ['id', 'c_3']]]]
```

or all of this can be inside a nested `if`/`while` statement, *etc...*